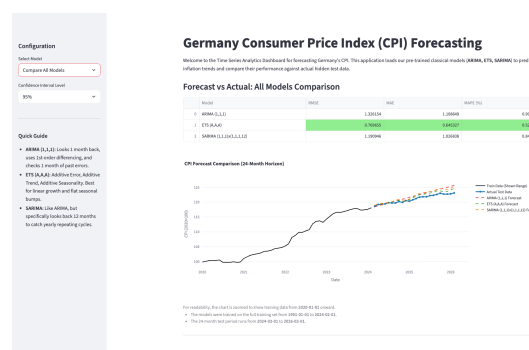


Forecasting Germany's Consumer Price Index (CPI) using Time Series Models



Author: Siddhanth Sharma
 Matriculation No.: 7027189
 Course of Studies: Business Intelligence and Data Analytics

Author: Nayan Chavan
 Matriculation No.: 7027933
 Course of Studies: Business Intelligence and Data Analytics

Author: Gargi Barve
 Matriculation No.: 7027883
 Course of Studies: Technical Management

First examiner: Prof. Dr. Elmar Wings
 Submission date: May 17, 2026

Contents

List of figures	iii
List of tables	v
1 Introduction	1
2 Quick Start	3
2.1 Recommended First Path	3
2.2 What to Check on the First Screen	4
2.3 What This Manual Covers Next	4
3 System Overview	5
3.1 Operational Goal	5
3.2 Normal Usage Sequence	5
3.3 Current Repository Result	6
4 Installation and First Launch	7
4.1 Prerequisites	7
4.2 Folder Check Before Launch	7
4.3 Normal Installation Sequence	7
4.3.1 Windows	7
4.3.2 macOS / Linux	8
4.4 Alternative Start Command	8
4.5 Expected First Launch Result	8
5 Dashboard User Interface	9
5.1 Layout Overview	9
5.2 Dashboard Layout Sketch	9
5.3 Dashboard Interface Screenshots	10
5.4 Interface Walkthrough Table	14
5.5 How to Read the Interface	14
6 Functions and Features	17
6.1 Select a Single Model	17
6.2 Compare All Models	17
6.3 Adjust the Confidence Interval Level	18
6.4 Use the Interactive Chart	18
6.5 Use the Legend	18

Contents

6.6	Download the Graph	18
6.7	Read Metrics and Chart Together	19
6.8	Use the Embedded Explanations	19
6.9	Practical Interpretation Rules	19
7	Help and Support	21
7.1	When to Ask for Support	21
7.2	Before Reporting an Issue	21
8	Troubleshooting	23
8.1	The Dashboard Does Not Start	23
8.2	The Browser Opens but No Forecast Appears	23
8.3	The Graph Appears but the Confidence Band Is Missing	23
8.4	The Wrong Page or Blank Page Opens	24
8.5	Legend or Chart Interaction Does Not Behave as Expected	24
8.6	Troubleshooting Decision Diagram	24
8.7	Quick Reference Table	25
9	Safety Guidelines	27
9.1	Interpretation Safety	27
9.2	Data Integrity Safety	27
9.3	Operational Safety	27
9.4	Dependency Safety	27
9.5	Privacy and Security Note	28
9.6	Usage Boundary	28
10	Specifications	29
10.1	System Scope	29
10.2	Input Specification	29
10.3	Output Specification	29
10.4	Supported User Controls	30
10.5	Dependency Inventory	30
10.6	Artifact Flow Specification	31
11	Maintenance	33
11.1	What End Users Should Know	33
11.2	What Maintainers Are Responsible For	33
11.3	Maintainer Timeline	33
11.4	User Boundary	35

List of Figures

3.1	Normal operating workflow for the CPI forecasting application.	6
5.1	Simplified layout sketch of the Streamlit dashboard.	10
5.2	Full dashboard view in the browser.	11
5.3	Sidebar close-up with the model selector, confidence interval selector, and quick guide.	12
5.4	Main dashboard area with forecast metrics and the interactive chart.	13
8.1	Troubleshooting path for the most common user-side problems. . . .	24
10.1	Artifact flow from raw CPI data to the deployed dashboard.	31
11.1	Maintainer-oriented upkeep timeline for the CPI dashboard (equally spaced 2-month intervals).	34

List of Tables

5.1	Dashboard interface elements and their user-facing purposes.	14
8.1	Common problems, causes, and user-side remedies.	25
10.1	Python dependencies required for operation.	30

1 Introduction

This manual explains how to operate the Germany Consumer Price Index (CPI) forecasting dashboard from a user perspective. The application is not a general-purpose data-science notebook or a developer toolkit. It is a focused Streamlit dashboard that lets the user inspect forecasts, compare forecasting models, adjust confidence intervals, and interpret the resulting graphs and evaluation metrics without editing Python source code.

The manual is written for three practical situations. First, it helps a first-time user launch the dashboard and confirm that the system is working correctly. Second, it guides a normal operator through the most important interface elements such as model selection, comparison mode, metric interpretation, and chart interaction. Third, it provides support-oriented sections for troubleshooting, operational safety, and technical reference so that the dashboard can be used confidently during demonstrations, project review, and day-to-day study work.

Welcome to the CPI Forecasting Dashboard Manual

This manual helps you start the dashboard, understand the controls, interpret the graph and metrics, and solve the most common user-side problems.

Key user goals:

- launch the dashboard correctly,
- compare ARIMA, ETS, and SARIMA views,
- adjust confidence intervals,
- use chart interactions such as legend selection and plot download.

The chapter order follows the normal user journey. The next chapter provides a quick-start path for users who want the shortest route to a successful first launch. After that, the manual explains the main function of the system, the installation and first-launch procedure, the dashboard interface, the available functions, help and support channels, troubleshooting, safety guidelines, technical specifications, and finally maintainer-oriented notes and optional extensions.

2 Quick Start

This chapter provides the shortest route from repository download to a working CPI dashboard. It is intended for users who want to start the application quickly and only then read the more detailed explanations in the later chapters.

2.1 Recommended First Path

For most users, the simplest workflow is:

1. open the [Code/](#) folder,
2. install the Python environment once,
3. start the dashboard,
4. open the local browser address shown by Streamlit,
5. select a model and inspect the forecast.

Quick Start for Windows

1. Open the [Code/](#) folder.
2. Run `setup_env.bat` once to install dependencies.
3. Run `run_dashboard.bat`.
4. Open the local browser address shown in the terminal, usually `http://localhost:8501`.

Expected result: the page title *Germany Consumer Price Index (CPI) Forecasting* appears together with a sidebar and a forecast chart.

Quick Start for macOS / Linux

1. Open a terminal in [Code/](#).
2. Run `./setup_env.sh` once to install dependencies.
3. Run `./run_dashboard.sh`.
4. Open the local browser address shown in the terminal.

Expected result: the dashboard opens with the configuration sidebar on the left and the forecast view on the right.

2.2 What to Check on the First Screen

After a successful launch, the user should see the following items:

- the title *Germany Consumer Price Index (CPI) Forecasting*,
- a sidebar with **Select Model** and **Confidence Interval Level**,
- either a single-model forecast view or a comparison view,
- metric output such as RMSE, MAE, and MAPE.

If these elements do not appear, continue with Chapter 4 and Chapter 8.

2.3 What This Manual Covers Next

The remaining chapters are organised according to practical user tasks. The manual first explains the purpose of the dashboard, then the installation and first launch, the user interface, the main dashboard functions, support channels, troubleshooting, safety guidelines, technical specifications, and maintainer-only upkeep notes.

3 System Overview

The main function of the system is to help a user inspect and compare classical time-series forecasts for Germany’s monthly Consumer Price Index. In practical terms, the software loads the official Destatis CPI history, reloads previously trained ARIMA, ETS, and SARIMA models, and displays their forecasts against the hidden 24-month test period inside an interactive Streamlit dashboard. The user does not have to train a model during every session, but the trained model files must exist before the dashboard can display results.

The current implementation is intentionally focused. It is not a general forecasting platform for arbitrary datasets. Instead, it is a project-specific application for the CPI series stored in [Code/data/61111-0002_en.csv](#). The dashboard exposes the user-visible decisions that are relevant in the repository state: model selection, comparison of all models, and selection of the confidence interval level. The forecast horizon is fixed in the code to 24 months, which keeps the evaluation and user interpretation consistent with the project report and the saved metrics in [Code/plots/evaluation_metrics.csv](#).

3.1 Operational Goal

From the operator’s perspective, the system answers three questions. First, how well do the three implemented forecasting approaches reproduce the CPI movement in the held-out test period? Second, which model currently performs best according to RMSE, MAE, and MAPE? Third, how wide is the forecast uncertainty when the user requests a stricter or looser confidence interval? The application therefore combines model loading, metric display, and visual interpretation in one interface rather than forcing the user to inspect raw console output.

3.2 Normal Usage Sequence

The intended user workflow is shown in Figure 3.1. Each step corresponds to a concrete repository artifact or command, which is important because the application depends on exact folder locations and pre-generated model files.

3 System Overview

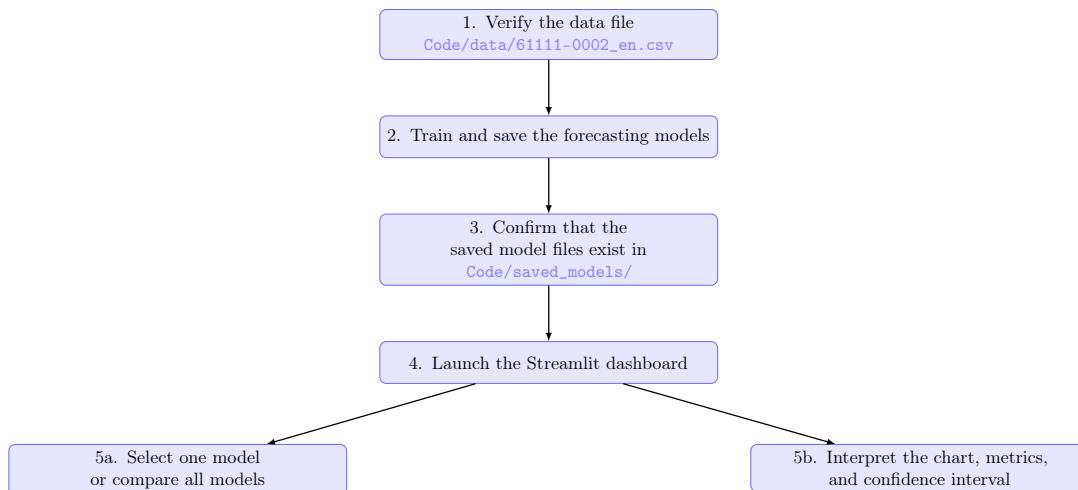


Figure 3.1: Normal operating workflow for the CPI forecasting application.

3.3 Current Repository Result

The saved evaluation file shows that ETS (A,A,A) is currently the strongest model in the repository state, with RMSE 0.7697, MAE 0.6453, and MAPE 0.5298%. ARIMA (1,1,1) and SARIMA (1,1,1) × (1,1,1,12) remain available because the application is designed for comparison rather than for single-model deployment only. This is useful for teaching, validation, and maintenance, because the user can immediately see whether a future data update changes the ranking.

The exact commands associated with Figure 3.1 are `python3src/modeling_pipeline.py` for model generation and `python3-mstreamlitrunapp.py` for the dashboard launch. Users who prefer helper scripts can instead run `./train_models.sh` and `./run_dashboard.sh` from the `Code/` directory.

Current Best Model

Based on the latest evaluation, **ETS (A,A,A)** is the strongest model in the repository state, with RMSE 0.7697, MAE 0.6453, and MAPE 0.5298%.

Why this matters: When you open the dashboard, ETS will likely track the actual CPI most closely. Use the comparison mode to see how ARIMA and SARIMA differ.

4 Installation and First Launch

This chapter explains how to prepare the software environment and start the CPI forecasting dashboard for normal use. The focus is the user-facing launch workflow, not the internal implementation of the forecasting pipeline.

4.1 Prerequisites

Before starting the dashboard, the user should ensure that the following are available:

- Python 3 with virtual-environment support,
- the repository folder including the [Code/](#) subdirectory,
- internet access for the initial dependency installation,
- a browser for opening the Streamlit dashboard.

4.2 Folder Check Before Launch

The most important working folder is [Code/](#). Before installation or launch, verify that the following items exist:

1. [Code/requirements.txt](#)
2. [Code/run_dashboard.bat](#) or [Code/run_dashboard.sh](#)
3. [Code/saved_models/](#)

If the saved model files are missing, the dashboard may still start but will not be able to display the expected forecasts. In that case, the issue should be treated as a maintainer-side problem and reported through the support process.

4.3 Normal Installation Sequence

4.3.1 Windows

For Windows users, the recommended first-run sequence is:

1. open the [Code/](#) folder,

4 Installation and First Launch

2. run `setup_env.bat`,
3. after installation completes, run `run_dashboard.bat`.

4.3.2 macOS / Linux

For macOS or Linux users, the recommended first-run sequence is:

1. open a terminal in `Code/`,
2. run `./setup_env.sh`,
3. after installation completes, run `./run_dashboard.sh`.

4.4 Alternative Start Command

The repository also provides `start.bat` and `start.sh`. These scripts are useful when the full environment and dashboard startup should be executed in one sequence. However, for normal day-to-day usage, the dedicated dashboard start scripts are easier to understand and are therefore preferred in this manual.

4.5 Expected First Launch Result

After a successful launch, Streamlit prints a local address in the terminal. Open that address in a browser. The first screen should show:

- the dashboard title,
- the configuration sidebar,
- forecast metrics,
- a forecast graph,
- explanatory sections below the main chart.

Important Usage Note

Normal users should **not** update the raw CPI file, retrain the forecasting models, or edit Python modules just to use the dashboard. If such actions become necessary, the issue belongs to a maintainer and should be handled through the support process described later in this manual.

5 Dashboard User Interface

This chapter introduces the visual structure of the CPI forecasting dashboard. The goal is to help the user understand where the controls are located, what each area of the interface does, and how the displayed output should be read during normal use.

5.1 Layout Overview

The dashboard uses a simple two-area structure:

- **Sidebar (left):** model selection, confidence interval selection, and quick guidance text,
- **Main content area (right):** title, metrics, interactive chart, and interpretation panels.

This arrangement supports a natural user flow: choose settings on the left and interpret the resulting output on the right.

5.2 Dashboard Layout Sketch

Figure 5.1 provides a simplified structural sketch of the dashboard.

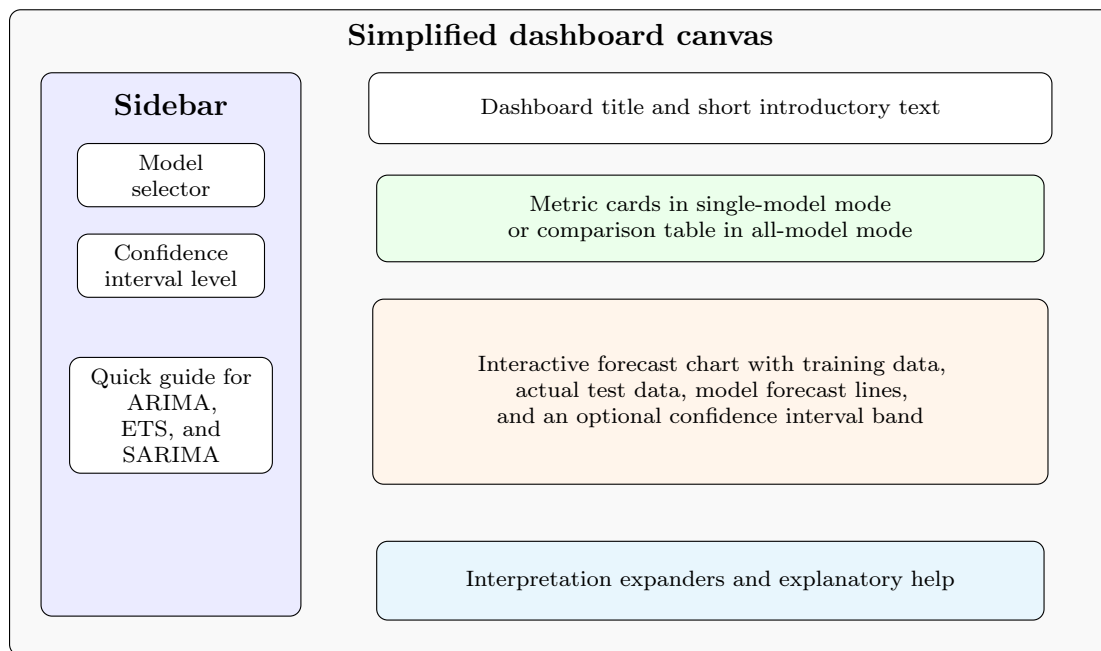


Figure 5.1: Simplified layout sketch of the Streamlit dashboard.

5.3 Dashboard Interface Screenshots

Figures 5.2, 5.3, and 5.4 document the running dashboard from the current repository state.

5.3 Dashboard Interface Screenshots

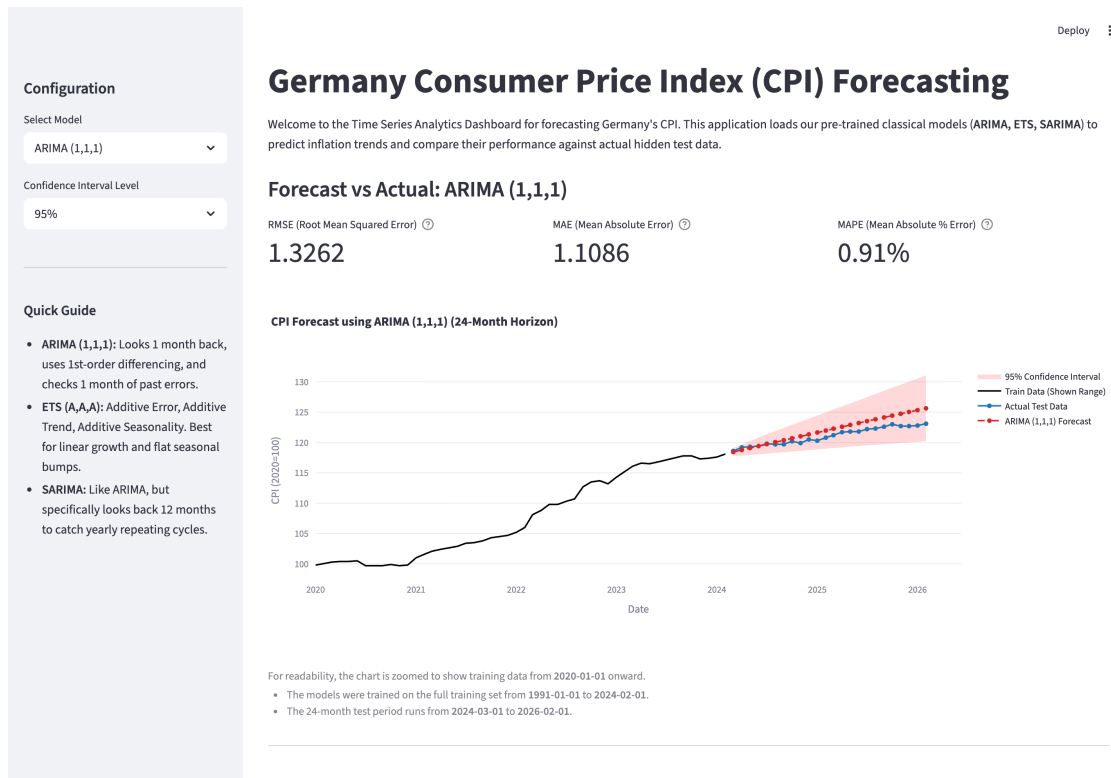


Figure 5.2: Full dashboard view in the browser.

5 Dashboard User Interface

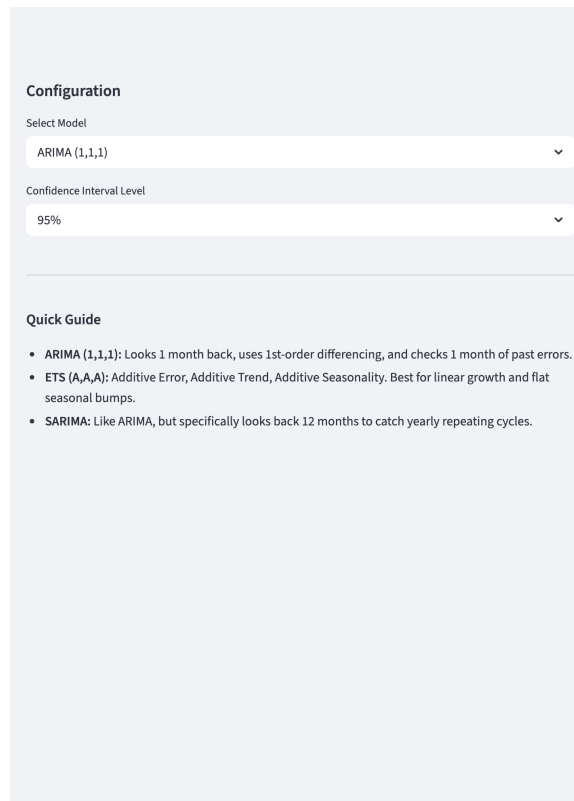


Figure 5.3: Sidebar close-up with the model selector, confidence interval selector, and quick guide.

Germany Consumer Price Index (CPI) Forecasting

Welcome to the Time Series Analytics Dashboard for forecasting Germany's CPI. This application loads our pre-trained classical models (ARIMA, ETS, SARIMA) to predict inflation trends and compare their performance against actual hidden test data.

Forecast vs Actual: ARIMA (1,1,1)

RMSE (Root Mean Squared Error) ⓘ

1.3262

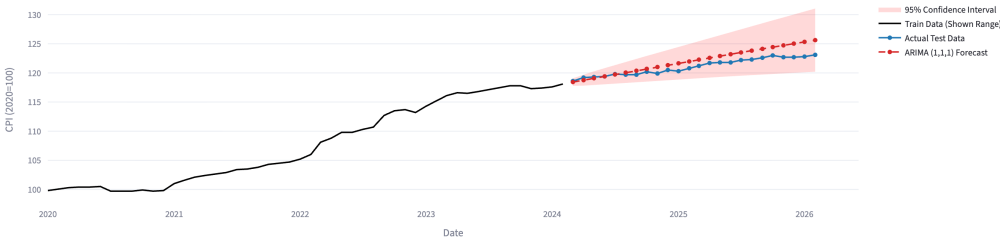
MAE (Mean Absolute Error) ⓘ

1.1086

MAPE (Mean Absolute % Error) ⓘ

0.91%

CPI Forecast using ARIMA (1,1,1) (24-Month Horizon)



For readability, the chart is zoomed to show training data from 2020-01-01 onward.

- The models were trained on the full training set from 1991-01-01 to 2024-02-01.
- The 24-month test period runs from 2024-03-01 to 2026-02-01.

Figure 5.4: Main dashboard area with forecast metrics and the interactive chart.

5.4 Interface Walkthrough Table

UI Element	Location	Purpose
Select Model	Sidebar	Chooses between ARIMA, ETS, SARIMA, and comparison mode.
Confidence Interval Level	Sidebar	Changes the width of the uncertainty band in single-model mode.
Quick Guide	Sidebar	Gives a short plain-language explanation of each forecasting model.
Metric cards	Main area	Show RMSE, MAE, and MAPE for the selected model.
Comparison table	Main area	Replaces the metric cards when all-model comparison mode is selected.
Interactive chart	Main area	Displays training data, actual test data, forecasts, and confidence intervals.
Legend	Right side of chart	Allows traces to be hidden or shown interactively.
Plot toolbar	Upper chart area	Provides zoom, pan, reset, fullscreen, and plot download actions.
Interpretation expanders	Lower main area	Explain the forecasting pipeline, metrics, and confidence intervals in user-friendly language.

Table 5.1: Dashboard interface elements and their user-facing purposes.

5.5 How to Read the Interface

The normal sequence is simple:

1. choose a model in the sidebar,
2. choose a confidence interval level,
3. read the metric area,
4. inspect the forecast chart,
5. open the explanation panels if more interpretation help is needed.

When **Compare All Models** is selected, the metric cards are replaced by a comparison table and the chart overlays all model forecasts in one view. When one specific

model is selected, the chart shows that model in detail together with its confidence interval band.

Dashboard Reading Tips

- **Black line:** training data (recent history shown for clarity)
- **Blue line with markers:** actual test data (the "ground truth" hidden from models)
- **Red dashed line:** model forecast
- **Shaded band:** confidence interval (wider = more uncertainty)

Tip: If the red line closely follows the blue line, the model is performing well.

6 Functions and Features

This chapter explains what the user can do with the dashboard after it has started successfully. The focus is the visible behaviour of the application rather than the internal implementation of the forecasting models.

6.1 Select a Single Model

The main user action is the model selector in the sidebar. The user can choose:

- ARIMA (1,1,1)
- ETS (A,A,A)
- SARIMA (1,1,1)x(1,1,1,12)

When one of these options is selected, the dashboard displays:

- three metric cards (RMSE, MAE, MAPE),
- one forecast chart,
- one confidence interval band,
- one legend describing the plotted lines.

This mode is useful when the user wants to inspect one forecast method in detail.

6.2 Compare All Models

The option **Compare All Models** changes the dashboard into comparison mode. In that mode, the application shows:

- a table with one row per model,
- an overlaid chart containing all three forecast curves,
- the actual test data for direct visual comparison.

This is the fastest way to see how differently the models behave on the same test period.

6.3 Adjust the Confidence Interval Level

For single-model views, the confidence interval selector allows the user to choose values from 80% to 99%. The selected value changes the width of the uncertainty band shown around the forecast line.

- A lower percentage produces a narrower band.
- A higher percentage produces a wider band.

This function changes the visual uncertainty display, but it does not retrain the model.

6.4 Use the Interactive Chart

The main chart supports several interactive actions that are useful during interpretation:

- hover over plotted values to inspect exact points,
- zoom into the chart,
- pan across the visible region,
- reset the chart to its default view,
- open the chart in fullscreen mode.

6.5 Use the Legend

The legend on the right side of the chart is not only descriptive; it is also interactive.

- Clicking a legend item hides the corresponding trace.
- Clicking the same item again shows the trace again.
- This allows the user to simplify the chart visually when multiple lines overlap.

This is especially useful in comparison mode, where the user may want to focus on only one or two forecasts at a time.

6.6 Download the Graph

The Plotly toolbar includes a download option labeled *Download plot as a PNG*. This allows the user to export the currently visible graph as an image for presentation or documentation purposes.

The exported file reflects the current visible state of the graph. If traces were hidden through the legend or the chart was zoomed before export, the downloaded image follows that visible state.

6.7 Read Metrics and Chart Together

The dashboard is designed so that numerical interpretation and visual interpretation support each other:

- the metric values summarize overall forecast quality,
- the chart reveals where that quality was strong or weak across time,
- the confidence interval adds a visible uncertainty range.

The user should therefore avoid reading the metrics in isolation. A low average error is useful, but the chart is still necessary to understand whether a model missed seasonal patterns or diverged late in the forecast horizon.

6.8 Use the Embedded Explanations

The lower part of the dashboard contains expandable explanation panels. These panels help users understand:

- how the training and test split works,
- what confidence intervals mean,
- how RMSE, MAE, and MAPE should be interpreted,
- why the compared models may behave differently.

6.9 Practical Interpretation Rules

To avoid misuse, the user should keep the following rules in mind:

Interpretation Cheat Sheet

1. **Lower RMSE / MAE / MAPE** = better forecast accuracy
2. **Wider confidence interval** = more conservative uncertainty, not better performance
3. **Dashboard purpose:** analytical comparison, not an official macroeconomic forecast
4. **Comparison mode:** best for seeing which model wins overall
5. **Single-model mode:** best for inspecting one model's behaviour in detail

7 Help and Support

This chapter explains what a user should do when the dashboard does not behave as expected and where support should be requested. The goal is to help the operator distinguish between simple local usage issues and situations that should be escalated to the project team.

7.1 When to Ask for Support

Support should be requested when the dashboard cannot be started, when expected model outputs are missing, when the browser page loads but does not display the expected controls or graphs, or when the visible results appear inconsistent with the documented workflow of this manual. Minor interaction questions, such as how to hide a legend item or download a chart image, are answered in the user-interface and functions chapters and should normally be checked there first.

Support Channels

- **Primary contact:** contact the project authors through the agreed course communication channel.
- **Repository support:** use the project GitHub repository for reproducible issue reporting when screenshots, logs, or file paths need to be shared.
- **What to include:** operating system, launch command used, visible error message, and a screenshot of the dashboard or terminal window.

7.2 Before Reporting an Issue

Before asking for support, the user should verify four things:

1. the command was executed from the intended directory,
2. the browser address shown by Streamlit was opened correctly,
3. the expected dashboard title and sidebar controls are absent or incorrect, not merely collapsed,
4. the problem also remains after restarting the dashboard once.

These checks reduce avoidable support traffic and make it easier for the project team to focus on genuine reproducible problems.

8 Troubleshooting

This chapter helps the user respond to the most common dashboard-side problems. The structure is symptom-oriented: first identify what went wrong, then apply the smallest reasonable fix.

8.1 The Dashboard Does Not Start

If the dashboard does not start after running the launch script, the most common causes are:

- dependencies were not installed,
- the script was started from the wrong folder,
- the virtual environment was not created correctly.

First, rerun the installation step from Chapter 4. If the problem remains, contact support.

8.2 The Browser Opens but No Forecast Appears

If the page title appears but no valid forecast is shown, the likely cause is that required model artifacts are missing or inconsistent. In that situation, a normal user should not attempt backend repair directly. Instead, document the visible message and report it to a maintainer.

8.3 The Graph Appears but the Confidence Band Is Missing

If the chart appears but the confidence interval does not, the dashboard may still be usable for basic point-forecast inspection. Try changing the model or confidence interval level once. If the problem remains, report it as a maintainer-side issue.

8.4 The Wrong Page or Blank Page Opens

If the browser opens a page that does not show the CPI dashboard title, confirm that the local Streamlit address was copied correctly from the terminal window. If multiple local pages are open in the browser, refresh the tab or reopen the exact address printed during launch.

8.5 Legend or Chart Interaction Does Not Behave as Expected

If clicking a legend item appears to do nothing, click it once more and watch the line visibility carefully. In dense charts, hidden traces may still seem close to the remaining lines. If zoom or pan behaves unexpectedly, use the chart reset control to return to the default view.

8.6 Troubleshooting Decision Diagram

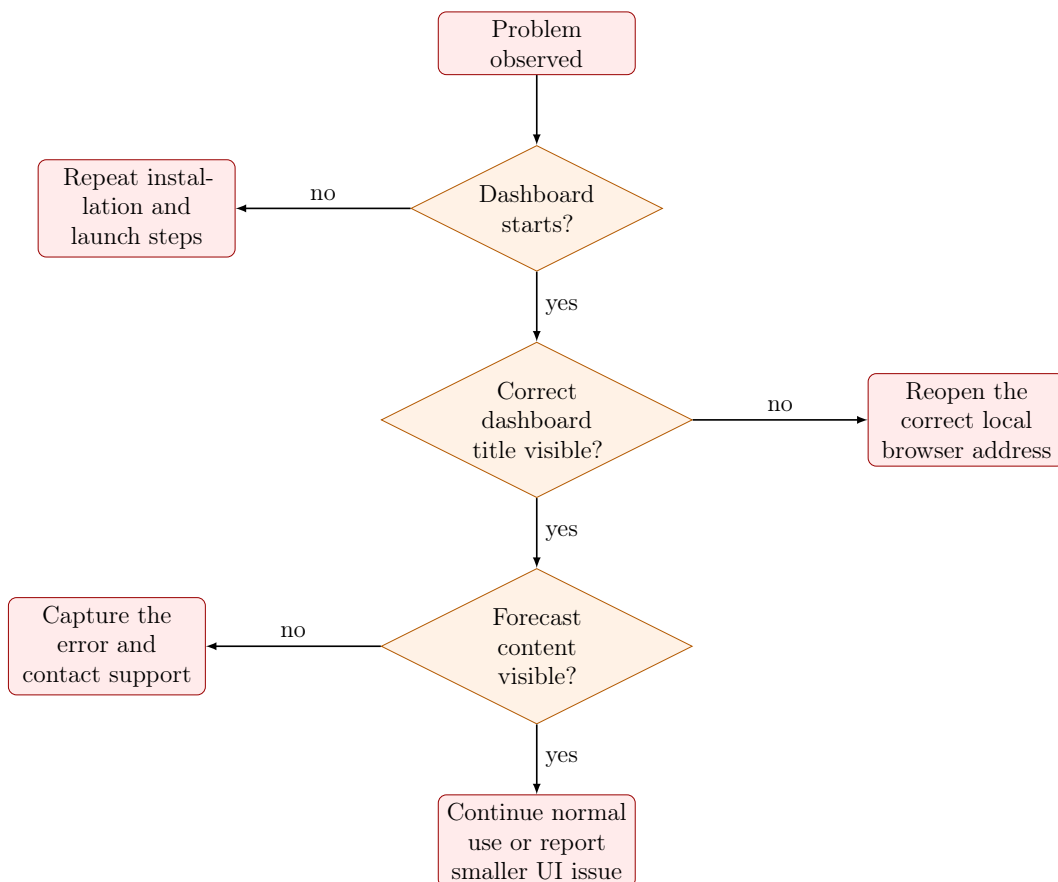


Figure 8.1: Troubleshooting path for the most common user-side problems.

8.7 Quick Reference Table

Table 8.1: Common problems, causes, and user-side remedies.

Symptom	Likely cause	Recommended action
Launch script fails	Environment or dependency issue	Repeat the installation steps and verify you are in Code/ .
Browser opens wrong page	Wrong local URL or old tab	Reopen the exact local address printed by Streamlit.
Dashboard loads but no forecast appears	Missing or broken model artifacts	Capture the message and contact a maintainer.
Confidence interval missing	Partial model-output issue	Try another model or interval level; if unchanged, report it.
Chart interaction seems odd	Zoom/legend state changed	Use the chart reset control and try again.

Quick Diagnostic Checklist

Before contacting support, verify:

- Did you run the launch script from the [Code/](#) directory?
- Did you open the exact local URL printed by Streamlit?
- Did you wait for the dashboard title and sidebar to load?
- Did you try refreshing the page or restarting the dashboard once?

If all four are true and the problem remains, it is time to escalate.

9 Safety Guidelines

This project does not control physical hardware, so the main safety concerns are operational rather than mechanical. The user should nevertheless follow a few rules to avoid incorrect interpretations, damaged repository state, or broken application sessions.

9.1 Interpretation Safety

The dashboard is an academic forecasting tool. Its outputs should be treated as analytical support for learning and comparison, not as an official macroeconomic forecast for policy or investment decisions. A visually convincing chart does not remove the uncertainty inherent in forecasting, especially when inflation regimes change.

9.2 Data Integrity Safety

Normal users should not replace the CPI CSV at all. If a data update is genuinely required, it should be handled by a maintainer using a documented workflow. Do not mix manually edited values into the official series, because the saved models and the reported metrics would then no longer describe the same dataset.

9.3 Operational Safety

Do not delete files from `Code/saved_models/` while the dashboard is being started or refreshed. Normal users should also avoid editing `Code/src/`, `Code/app.py`, or backend data files merely to solve a usage problem. If the dashboard does not behave as expected, consult Chapter 8 or report the issue through the support channel.

9.4 Dependency Safety

Install packages inside the dedicated virtual environment when possible. System-wide package installation can make troubleshooting harder because the application may then depend on an uncontrolled combination of Python libraries.

9.5 Privacy and Security Note

The project uses public macroeconomic data and therefore has no direct personal-data risk in normal use. The main protection goal is reproducibility: users should preserve the integrity of the repository structure, saved models, and evaluation outputs.

9.6 Usage Boundary

The safe operating boundary for a normal user is simple: launch the dashboard, use the dashboard, interpret the visible output, and export the graph if needed. Actions beyond that boundary, such as retraining models, replacing source data, or modifying code, belong to maintainers rather than end users.

Safety at a Glance

- **Do not** treat dashboard forecasts as official policy or investment advice.
- **Do not** replace the CPI CSV or edit backend code unless you are a maintainer.
- **Do not** delete files from `Code/saved_models/` while the dashboard is running.
- **Do** install dependencies inside the dedicated virtual environment.
- **Do** preserve repository integrity so results remain reproducible.

10 Specifications

This chapter records the supported operating assumptions of the current dashboard release. It is intended as a compact technical reference for users who want to understand the scope of the application without reading backend source code.

10.1 System Scope

The application supports one forecasting use case: monthly CPI forecasting for Germany using the Destatis table 61111-0002. The data is used as a univariate time series, and the implemented model inventory consists of ARIMA (1,1,1), ETS (A,A,A), and SARIMA (1,1,1) $\times$ (1,1,1,12).

10.2 Input Specification

- **Primary input file:** [Code/data/61111-0002_en.csv](#)
- **Expected frequency:** monthly
- **Expected target variable:** CPI index value with base year 2020=100
- **Observed usable date range in the current file:** January 1991 to February 2026
- **Usable observations after dropping future placeholders:** 422 monthly values

The data loader removes future placeholder entries such as . . . , maps month names to numeric values, and enforces a strict month-start frequency.

10.3 Output Specification

For normal dashboard use, the following prepared artifacts are expected to be present.

- Serialized model files in [Code/saved_models/](#)
- Plot images in [Code/plots/](#)
- Evaluation metrics in [Code/plots/evaluation_metrics.csv](#)
- A LaTeX version of the metrics table in [Code/plots/evaluation_metrics.tex](#)

10.4 Supported User Controls

The dashboard currently supports the following interactive controls and user actions.

- single-model selection
- all-model comparison mode
- confidence interval level selection from 80% to 99%
- legend-based hiding and showing of plotted traces
- plot download through the built-in chart toolbar

The forecast horizon is not user-configurable in the present implementation. It is fixed to the 24-month test window used in both the training pipeline and the dashboard logic.

Scope Reminder

This dashboard is intentionally focused on one dataset and three classical models. It does **not** support arbitrary datasets, custom forecast horizons, or real-time data feeds. If you need those capabilities, refer to the Add Ons chapter for planned extensions.

10.5 Dependency Inventory

The runtime dependencies declared in [Code/requirements.txt](#) are listed in Table 10.1.

Table 10.1: Python dependencies required for operation.

Package	Role in the project
pandas	data loading, date parsing, indexing, and table handling
matplotlib	generation of saved forecast and decomposition plots
statsmodels	ARIMA, ETS, and SARIMA implementations
scikit-learn	RMSE, MAE, and MAPE evaluation metrics
jinja2	dependency required by the current software environment
joblib	serialization and loading of trained model objects
streamlit	web application framework for the dashboard
plotly	interactive chart rendering in the dashboard

10.6 Artifact Flow Specification

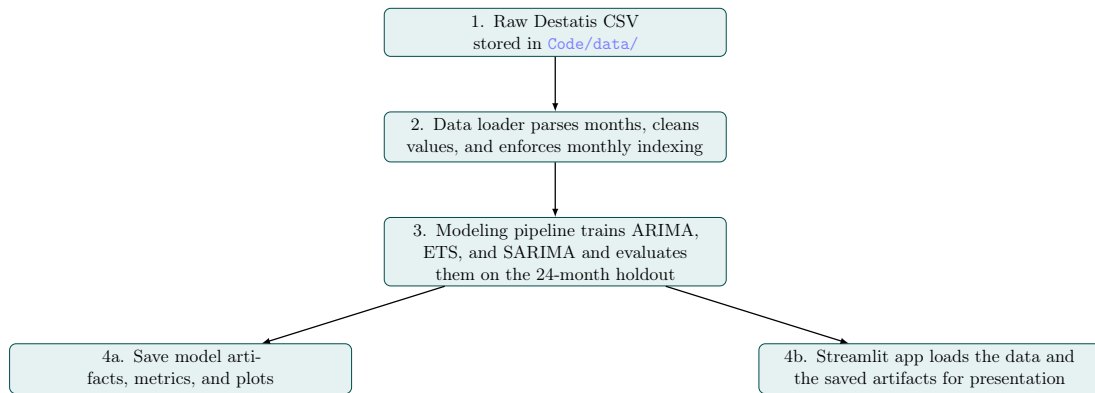


Figure 10.1: Artifact flow from raw CPI data to the deployed dashboard.

11 Maintenance

This chapter is intended primarily for maintainers, supervisors, or team members who are responsible for keeping the repository operational over time. Normal end users do not need to perform the actions described here in order to launch and use the dashboard.

11.1 What End Users Should Know

For normal dashboard use, no data update, model retraining, or backend editing is required. If the dashboard opens and shows valid forecasts, the user can continue working without touching the repository internals.

If the dashboard fails because data or model artifacts are missing, the correct action for a normal user is to contact a maintainer rather than modifying backend files independently.

11.2 What Maintainers Are Responsible For

Maintainers may occasionally need to:

- verify that the saved model files still exist,
- confirm that dependency installation still succeeds,
- refresh or repair the repository state if files become corrupted,
- update screenshots or documentation if the user interface changes visibly.

This chapter deliberately does not encourage casual end users to replace the CPI source file or retrain the forecasting backend without coordination.

11.3 Maintainer Timeline

Figure 11.1 shows a realistic maintainer-oriented upkeep timeline for the CPI forecasting dashboard. The intervals are designed for a project that uses monthly macroeconomic data and requires periodic validation of both the software environment and the model accuracy.

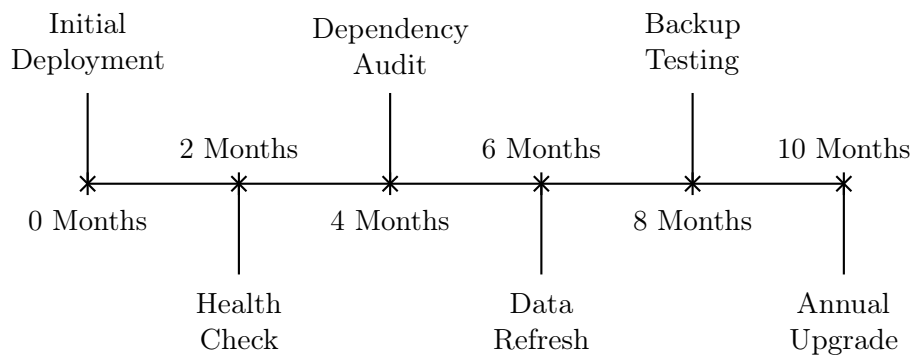


Figure 11.1: Maintainer-oriented upkeep timeline for the CPI dashboard (equally spaced 2-month intervals).

Explanation of milestones. The timeline above reflects a practical maintenance rhythm for a forecasting application that relies on monthly macroeconomic data. All events are spaced at equal 2-month intervals to create a predictable, easy-to-remember schedule:

- **Initial Deployment (Month 0):** Train all three models (ARIMA, ETS, SARIMA) on the latest available CPI data, save the serialized artifacts, and confirm that the Streamlit dashboard launches correctly.
- **Health Check (Month 2):** Verify that the dashboard still loads after the first two months of real-world use. Compare the model’s two-month-ahead prediction against the newly released Destatis CPI values to detect any early drift.
- **Dependency Audit (Month 4):** Review [Code/requirements.txt](#) for outdated packages. Update patch-level versions (e.g., 1.2.3 → 1.2.4) and test that the dashboard still compiles and runs. Avoid major version jumps unless necessary.
- **Data Refresh (Month 6):** Download the newest Destatis CPI release, append it to the dataset, and retrain the models. Evaluate the new models against the same 24-month holdout to ensure performance has not degraded. Save new artifacts and update screenshots if the UI changed.
- **Backup Testing (Month 8):** Confirm that the repository can be cloned to a fresh machine and that the setup scripts ([setup_env.sh](#) or [setup_env.bat](#)) still execute without errors. This validates reproducibility.
- **Annual Upgrade (Month 10):** Perform a full dependency upgrade cycle, review the manual and report for outdated claims, and consider whether any Add Ons (see Chapter ??) should be implemented in the next development cycle.

For a student project that is no longer actively developed, the maintainer can treat Months 0–4 as the most critical validation window. If the dashboard passes the initial deployment, health check, and dependency audit during that period, the longer-term milestones become optional rather than mandatory.

11.4 User Boundary

End users should not:

- edit `Code/src/` files,
- replace the CPI CSV casually,
- delete `Code/saved_models/` contents,
- rerun backend retraining unless explicitly instructed by a maintainer.

Keeping this boundary clear reduces accidental breakage and makes the dashboard safer to use in demonstrations and review situations.

User vs Maintainer Boundary

Normal users: launch the dashboard, select models, compare forecasts, interpret metrics, download charts, and report issues.

Maintainers only: update data, retrain models, repair artifacts, modify code, and refresh documentation.

If you are unsure which role you have, assume you are a normal user.

 Contact for Technical Assistance

For any technical questions or troubleshooting support, please contact the development team:

- Siddhanth Sharma – siddhanth.sharma@stud.hs-emden-leer.de
- Nayan Chavan – nayan.chavan.1@stud.hs-emden-leer.de
- Gargi Barve – gargi.nilesh.barve@stud.hs-emden-leer.de

Response time: typically within **2–3 working days**.